

The 0/1 Knapsack problem

Knapsack visualization:

How to:

før du kører **start** skal der indtastes tal i "max capacity" og genereres items

Animation speed:

Items:

Item 0 vægt: 4, værdi: 5
Item 1 vægt: 3, værdi: 9
Item 2 vægt: 5, værdi: 18
Item 3 vægt: 1, værdi: 17
Item 4 vægt: 1, værdi: 18

Recursion Tree:

iteration: 16 1) Items: [2,3,4] with Value: 53 and TotalWeight: 7 2) Items: [3,4] with Value: 35 and TotalWeight: 2 <i>Chosen: 1</i>
iteration: 17 1) Items: [1,3,4] with Value: 44 and TotalWeight: 5 2) Items: [2,3,4] with Value: 53 and TotalWeight: 7 <i>Chosen: 2</i>
iteration: 18 1) Items: [0,3,4] with Value: 40 and TotalWeight: 6 2) Items: [2,3,4] with Value: 53 and TotalWeight: 7 <i>Chosen: 2</i>

Tabel:

11	1	2	3	4	5	18	1
12	item: 1	item: 2	item: 3	item: 4	item: 5	35	2
13	item: 1	item: 2	item: 3	item: 4	item: 5	18	1
14	item: 1	item: 2	item: 3	item: 4	item: 5	18	1
15	item: 1	item: 2	item: 3	item: 4	item: 5	35	2
16	item: 1	item: 2	item: 3	item: 4	item: 5	53	7
17	item: 1	item: 2	item: 3	item: 4	item: 5	53	7
18	item: 1	item: 2	item: 3	item: 4	item: 5	53	7

Antal repetitioner: 18
Maksimal værdi: 53
Total vægt: 7
Valgte items: Item 2, Item 3, Item 4

Gruppe:

Rolf Lundsgaard Josephsen

Github Repo:

<https://github.com/mrPrimeBeef/Knapsack-problem>

Deployed version:

<https://mrprimebeef.github.io/Knapsack-problem/>

Indholdsfortegnelse

Indholdsfortegnelse.....	2
Beskrivelse af Knapsack-problemet.....	3
Visualiseringen.....	3
Datastrukturer.....	3
Algoritme.....	4
Big O-notation.....	4
Tidskompleksitet: $O(2^n)$	4
Rumkompleksitet: $O(n)$	4
Pseudokode for Knapsack algoritme.....	5
Implementationen.....	6
Links:.....	6

Beskrivelse af Knapsack-problemet

the 0/1 Knapsack problemet handler om at vælge genstande til en rygsæk, når der ikke er plads til alt. Hver genstand har både en vægt og en værdi, og opgaven er at finde den kombination af genstande, som giver den højeste samlede værdi uden at overskride rygsækkens kapacitet.

Denne algoritme gennemgår alle de forskellige kombinationer af genstande for at finde den bedste løsning. Den anvender "divide and conquer"-teknikken, som systematisk opdeler problemet ved at teste forskellige muligheder. ligesom ved mergesort, der deler input i mindre dele til det når et "base case".

Visualiseringen

I visualiseringen kan man generere 5 items, som hver har en vægt og en værdi. Disse bliver genereret tilfældigt, hvor vægtene ligger mellem 1-5 og en max kapacitet er som standard sat 7, hvilket sikrer en afbalanceret fordeling, hvor ikke alle items automatisk kommer i rygsækken. Men du har mulighed for selv at sætte den og generere nye items. Værdierne ligger mellem 1-20. Der er også mulighed for at justere animationshastigheden fra 100ms til 2000ms per iteration. Det sat sådan for mindst mulig configuration for at køre algoritmen.

Når visualiseringen kører, kan man se to forskellige repræsentationer af algoritmen samtidigt:

Rekursionstræet viser hver iteration hvilke items der bliver holdt op mod hinanden, hvad der bliver inkluderet eller skippet, hvor værdi og vægt beregnes for begge muligheder. Fordi algoritmen er rekursiv, starter træet nede ved base cases og arbejder sig bagud, hvor hvert resultat sammenlignes efterhånden som de rekursive kald returneres fra stakken.

Tabellen viser de samme data, men i en mere struktureret form. Hver række repræsenterer en iteration, og kolonner viser hvilke items der blev valgt, samt den samlede værdi og vægt. Dette giver et andet overblik over hvordan algoritmen kører.

Resultatet vises tilsidst som både numeriske værdier (maksimal værdi, total vægt, antal iterationer) og visuelt ved at markere de valgte items med en grøn baggrund. På denne måde kan man se både hvordan algoritmen arbejder, og hvad det endelige optimale valg er

Datastrukturer

Arrays: Bruges til at gemme listen af items (items), valgte items (selectedItems), og alle træ-beslutninger (treeLog).

Objekter: Bruges til at repræsentere items (med *weight* og *value* egenskaber) og til at holde styr på resultaterne fra hver rekursiv beregning (med *maxValue*, *selectedItems*, *totalWeight*)

Algoritme

Big O-notation

Kompleksiteten for denne algoritme når man løser den rekursivt er $O(2^n)$, fordi hvert item har to muligheder, at inkludere eller udelukke, hvilket skaber 2^n kombinationer i værste tilfælde. Dette gør metoden ineffektiv for større datasæt.

Tidskompleksitet: $O(2^n)$

- Med 5 elementer: ~32 operationer
- Med 10 elementer: ~1.024 operationer

Rum Kompleksitet: $O(n)$

- Hvert funktionskald bruger $O(1)$ plads
- Funktionen skal kaldes n gange, hvor n er dybden af stakken
- Selvom algoritmen køres 2^n gange, gemmes der kun $O(n)$ variabler i hukommelsen på samme tid, fordi der kun er én aktiv funktionskald per dybde på call stacken

Rekursiv løsning bruger mindre plads end iterative løsninger, men kører langsommere. Den sparede tid i den iterative tilgang opnås ved at bruge mere hukommelse til at gemme "mellemresultater".

Algoritmen implementerer 0/1 Knapsack-problemet gennem eksponentiel rekursiv søgning med Divide and Conquer princippet. Den fungerer ved at:

1. Dele problemet op i to valg for hvert item: enten inkludere det eller ekskludere det
2. Rekursivt løse de mindre delproblemer
3. Sammenligne de to valgmuligheder og vælge den med højest værdi

Rekursionstræet: Algoritmen bygger et binært træ mens den udforsker løsningsrummet, hvor hver node repræsenterer en beslutning om et specifikt item. Base cases vises først (når vi når til slutningen af items-listen eller når kapaciteten er fyldt), og træet ekspanderer bagud til "root-noden".

Pseudokode for Knapsack algoritme

- 1) man checker for om man er nået base case (sidste item eller kapacitet = 0)
- 2) Hvis det nuværende item passer i rygsækken
 - a) Gem det item og beregn resultatet rekursivt med mindre kapacitet
 - b) Dette er first choice (inkluder item)
- 3) uanset hvad(ikke else): prøv at beregne hvordan det ville se ud, uden dette item (skip item)
 - a) Beregn resultatet rekursivt med samme kapacitet
 - b) Dette er second choice
- 4) så sammenligner man hvilket choice der har størst værdi
- 5) Returner den bedste løsning

KnapsackRecursive (items, max capacity, index)

Null check

// base case: Stop når vi ikke har flere items eller kapacitet
Base case check

currentItem = items[index]

if

currentItem weight <= max capacity

```
result = KnapsackRecursive
(
  items
  max capacity - currentItem weight
  index +1
)
firstChoice = value + result value
              weight + result weight
              items + item
```

else

// currentItem skippes - så den skal bare kalde funktionen igen med det næste index

```
secondChoice = KnapsackRecursive
(
  items
  max capacity
  index +1
)
```

biggest = max(firstChoice, secondChoice)

return biggest

Implementationen

Til at starte med bliver knapsack kaldt og ikke knapsackRecursive nu kendt som ksr, fordi ksr har brug for ekstra attributter, som brugeren af knapsack ikke bekymre sig om. derudover er der en global count og treeLog/Logs variable som kun bruges til visualiseringen og ikke til algoritmen, count står for antal repetitioner og treeLog/logs er der hvor jeg gemmer de beslutninger der er taget.

first choice variablen bliver initialiseret ud af if statementet, så jeg kan tilgå det ud fra når jeg pusher til mig treeLog/logs. Den måde jeg opdateret first choice på er at bruge javascripts "spread operator", [index, ...result.selectedItems] bruges fordi JavaScript objekter er references. Uden "spread operator" ville alle løsninger pege på det samme array, hvilket ville give forkerte resultater. Med "spread operator" oprettes en ny kopi af arrayet. Der bliver brugt "post-order-travsel" til at logge "beslutningerne".

Links:

- <https://www.geeksforgeeks.org/dsa/0-1-knapsack-problem-dp-10/>
- <https://www.geeksforgeeks.org/dsadouble-knapsack-dynamic-programming/>
- <https://www.geeksforgeeks.org/dsa/unbounded-knapsack-repetition-items-allowed/>
- <https://www.youtube.com/watch?v=cJ21moQpofY&t=585s>
- <https://compgeek.co.in/branch-and-bound-algorithm/>
- <https://skilled.dev/images/post-order-traversal.gif>